Modul Praktikum: Access Modifier protected pada Java dengan Studi Kasus CRUD Mahasiswa

Capaian Pembelajaran

Setelah mengikuti praktikum ini, mahasiswa diharapkan mampu:

1. Menjelaskan fungsi dan batas akses dari private, default (tanpa kata kunci), protected, dan public.
2. Membuat dua package atau lebih, lalu menghubungkan class di dalamnya dengan import dan extends.
3. Menggunakan protected agar field dan method di class induk dapat dipakai class turunan yang berada di package berbeda.
4. Menyusun operasi CRUD sederhana (Create, Read, Update, Delete) memakai ArrayList.
5. Membuat tampilan grafis dengan JFrame, JTextField, JButton, dan JTable, lalu menghubungkannya dengan logika di class induk.
6. Menjelaskan alasan JFrame dipakai sebagai field (komposisi) dan tidak diwarisi langsung.

Gambaran Praktikum

Praktikum ini memakai tiga class yang disimpan dalam tiga package.

Package	Class	Peran
induk	Mahasiswa	Class utama. Menyimpan data dan semua operasi CRUD.
anak	MahasiswaApp	Class turunan dengan tampilan console berbasis menu.
gui	MahasiswaGUI	Class turunan dengan tampilan grafis (JFrame dan JTable).

Kedua class turunan tidak menulis ulang logika CRUD. Keduanya hanya memanggil method milik Mahasiswa. Inilah inti dari praktik protected: logika disimpan di satu tempat, dan hanya class turunan yang diberi izin memakainya dari luar package.

1. Dasar Teori

1.1 Access modifier

Access modifier menentukan siapa yang boleh mengakses sebuah field, method, atau constructor. Tujuannya menjaga agar bagian internal sebuah class tidak diubah sembarangan dari luar.

Modifier	Class sendiri	Package sama	Turunan di package lain	Class lain di package lain
private	Ya	Tidak	Tidak	Tidak
default	Ya	Ya	Tidak	Tidak
protected	Ya	Ya	Ya, lewat pewarisan	Tidak
public	Ya	Ya	Ya	Ya

1.2 Cara kerja protected lintas package

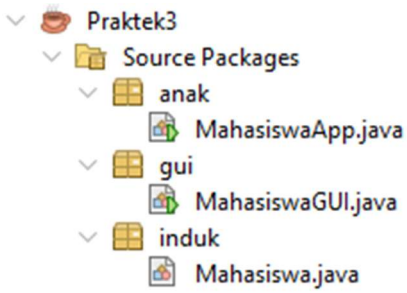
Di package yang sama, protected berperilaku seperti default. Perbedaannya baru terasa ketika class turunan berada di package lain. Pada kondisi itu, member protected hanya bisa dipakai melalui objek dari class turunan itu sendiri, baik secara langsung (misalnya tambahData(...)), lewat this, maupun lewat objek bertipe turunan.

Konsekuensinya, jika di dalam class MahasiswaGUI Anda menulis new Mahasiswa() lalu memanggil tambahData pada objek tersebut, kompilator akan menolaknya. Objek bertipe Mahasiswa dianggap sebagai objek "orang luar" bagi package gui. Aturan ini tercantum pada Java Language Specification bagian 6.6.2.

1.3 Pewarisan tunggal dan komposisi

Java hanya mengizinkan satu class induk. MahasiswaGUI sudah mewarisi Mahasiswa, sehingga tidak bisa sekaligus mewarisi JFrame. Solusinya, JFrame dibuat sebagai field di dalam MahasiswaGUI. Pola ini disebut komposisi: sebuah class memiliki objek lain, bukan menjadi turunannya.

2. Struktur Proyek



Nama folder harus sama dengan nama package, dan nama file harus sama dengan nama class public di dalamnya. Dua hal ini adalah sumber error paling sering pada pemula.

3. Class Induk: Mahasiswa

File: induk/Mahasiswa.java

```
package induk;

import java.util.ArrayList;

public class Mahasiswa {

    private String namaApp = "Aplikasi CRUD Mahasiswa";

    protected ArrayList<String> daftarNama = new ArrayList<>();

    public String getNamaApp() {
        return namaApp;
    }

    protected void tambahData(String nama) {
        daftarNama.add(nama);
        System.out.println("Data '" + nama + "' berhasil ditambahkan.");
    }

    protected void tampilData() {
        if (daftarNama.isEmpty()) {
            System.out.println("Data masih kosong.");
            return;
        }
        System.out.println("---- Daftar Mahasiswa ----");
        for (int i = 0; i < daftarNama.size(); i++) {
            System.out.println((i + 1) + ". " + daftarNama.get(i));
        }
    }

    protected boolean ubahData(int nomor, String namaBaru) {
        if (!nomorValid(nomor)) {
            return false;
```

```
        }
        daftarNama.set(nomor - 1, namaBaru);
        return true;
    }

    protected boolean hapusData(int nomor) {
        if (!nomorValid(nomor)) {
            return false;
        }
        daftarNama.remove(nomor - 1);
        return true;
    }

    protected boolean nomorValid(int nomor) {
        return nomor >= 1 && nomor <= daftarNama.size();
    }
}
```

Penjelasan

- namaApp bersifat private. Nilainya hanya bisa dibaca dari luar melalui getNamaApp() yang public. Ini contoh pola getter: data disembunyikan, akses dibuka secukupnya.
- daftarNama bersifat protected. Class turunan boleh membaca isinya, misalnya untuk menghitung jumlah data atau mengisi tabel, tetapi class lain di luar package tidak bisa menyentuhnya.
- Method tambahData, tampilData, ubahData, dan hapusData bersifat protected. Keempatnya mewakili Create, Read, Update, dan Delete.
- nomorValid adalah method bantu. Dengan menjadikannya protected, class turunan juga bisa memakainya untuk memeriksa nomor sebelum memanggil method lain.
- Nomor yang dipakai pengguna dimulai dari 1, sedangkan indeks ArrayList dimulai dari 0. Karena itu di dalam method selalu ada pengurangan nomor - 1.
- Method ubahData dan hapusData mengembalikan boolean agar pemanggil tahu apakah operasi berhasil atau nomor yang diberikan tidak valid.

4. Class Turunan Versi Console: MahasiswaApp

File: anak/MahasiswaApp.java

```
package anak;

import induk.Mahasiswa;
import java.util.Scanner;

public class MahasiswaApp extends Mahasiswa {

    private Scanner input = new Scanner(System.in);

    public void jalankan() {
        System.out.println("=== " + getNamaApp() + " ===");
```

```
        int pilihan;
        do {
            System.out.println("\nMENU");
            System.out.println("1. Tambah Data (Create)");
            System.out.println("2. Lihat Data (Read)");
            System.out.println("3. Ubah Data (Update)");
            System.out.println("4. Hapus Data (Delete)");
            System.out.println("0. Keluar");
            System.out.print("Pilih: ");
            pilihan = Integer.parseInt(input.nextLine());

            switch (pilihan) {
                case 1:
                    System.out.print("Nama mahasiswa: ");
                    String nama = input.nextLine();
                    tambahData(nama);
                    break;

                case 2:
                    tampilData();
                    System.out.println("Total data: " + daftarNama.size());
                    break;

                case 3:
                    tampilData();
                    System.out.print("Nomor yang diubah: ");
                    int noUbah = Integer.parseInt(input.nextLine());
                    System.out.print("Nama baru: ");
                    String namaBaru = input.nextLine();
                    if (ubahData(noUbah, namaBaru)) {
                        System.out.println("Data berhasil diubah.");
                    } else {
                        System.out.println("Nomor tidak valid.");
                    }
                    break;

                case 4:
                    tampilData();
                    System.out.print("Nomor yang dihapus: ");
                    int noHapus = Integer.parseInt(input.nextLine());
                    if (hapusData(noHapus)) {
                        System.out.println("Data berhasil dihapus.");
                    } else {
                        System.out.println("Nomor tidak valid.");
                    }
                    break;

                case 0:
                    System.out.println("Program selesai.");
                    break;

                default:
                    System.out.println("Pilihan tidak tersedia.");
            }
```

```
        } while (pilihan != 0);
    }

    public static void main(String[] args) {
        MahasiswaApp app = new MahasiswaApp();
        app.jalankan();
    }
}
```

Penjelasan

- Tanpa kata extends Mahasiswa, semua pemanggilan tambahData, tampilData, dan seterusnya akan gagal compile, karena class ini berada di package anak dan Mahasiswa berada di package induk.
- Pemanggilan tambahData(nama) tidak memakai nama objek di depannya. Method tersebut dipanggil pada objek MahasiswaApp itu sendiri, yang memang mewarisi method tersebut.
- Perintah daftarNama.size() menunjukkan bahwa field protected juga dapat dibaca langsung oleh class turunan.
- Program berjalan dalam perulangan do-while sampai pengguna memilih 0.
- Integer.parseInt(input.nextLine()) akan menimbulkan NumberFormatException bila pengguna memasukkan huruf atau menekan Enter tanpa angka. Untuk praktikum tingkat dasar, hal ini boleh dibiarkan dan cukup diketahui sebagai keterbatasan program.

5. Class Turunan Versi GUI: MahasiswaGUI

File: gui/MahasiswaGUI.java

```
package gui;

import induk.Mahasiswa;
import javax.swing.*;
import javax.swing.table.DefaultTableModel;
import java.awt.event.KeyAdapter;
import java.awt.event.KeyEvent;

/* paraktek ini menggunakan Jframe */

public class MahasiswaGUI extends Mahasiswa {

    private JFrame frame;
    private JTextField txtNama;
    private JTable tabel;
    private DefaultTableModel modelTabel;

    public MahasiswaGUI() {
        buatTampilan();
        muatTabel();
```

```
    }

    private void buatTampilan() {
        frame = new JFrame(getNamaApp()); // ambil dari method parent
        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        frame.setSize(500, 440);
        frame.setLocationRelativeTo(null);
        frame.setResizable(false);
        frame.setLayout(null); // layout manual pakai setBounds

        JLabel lblNama = new JLabel("Nama:");
        lblNama.setBounds(20, 20, 60, 25);
        frame.add(lblNama);

        txtNama = new JTextField();
        txtNama.setBounds(80, 20, 380, 25);
        frame.add(txtNama);

        // Tombol
        JButton btnTambah = new JButton("Tambah");
        btnTambah.setBounds(80, 60, 100, 30);
        frame.add(btnTambah);

        JButton btnUbah = new JButton("Ubah");
        btnUbah.setBounds(190, 60, 100, 30);
        frame.add(btnUbah);

        JButton btnHapus = new JButton("Hapus");
        btnHapus.setBounds(300, 60, 100, 30);
        frame.add(btnHapus);

        // Grid dengan komponen JTable
        // Kolom pada JTable harus dibentuk dahulu
        modelTabel = new DefaultTableModel(new String[]{"No", "Nama Mahasiswa"}, 0) {
            @Override
            public boolean isCellEditable(int row, int column) {
                return false; // sel tidak bisa diedit langsung
            }
        };

        tabel = new JTable(modelTabel);
        tabel.setSelectionMode(ListSelectionModel.SINGLE_SELECTION);
        tabel.setRowHeight(24);
        tabel.getColumnModel().getColumn(0).setMaxWidth(50);

        JScrollPane scroll = new JScrollPane(tabel);
        scroll.setBounds(20, 110, 440, 270);
        frame.add(scroll);

        // Event
        btnTambah.addActionListener(e -> aksiTambah());
        btnUbah.addActionListener(e -> aksiUbah());
        btnHapus.addActionListener(e -> aksiHapus());

        // klik baris -> nama tampil di textfield
```

```
        tabel.getSelectionModel().addListSelectionListener(e -> {
            int baris = tabel.getSelectedRow();
            if (!e.getValueIsAdjusting() && baris >= 0) {
                txtNama.setText(modelTabel.getValueAt(baris, 1).toString());
            }
        });

        // tekan tombol Delete di keyboard -> hapus baris terpilih
        tabel.addKeyListener(new KeyAdapter() {
            @Override
            public void keyPressed(KeyEvent e) {
                if (e.getKeyCode() == KeyEvent.VK_DELETE) {
                    aksiHapus();
                }
            }
        });
    }

    // baca isi grid dari daftarNama
    private void muatTabel() {
        modelTabel.setRowCount(0);
        for (int i = 0; i < daftarNama.size(); i++) {
            modelTabel.addRow(new Object[]{i + 1, daftarNama.get(i)});
        }
    }

    private void aksiTambah() {
        String nama = txtNama.getText().trim();
        if (nama.isEmpty()) {
            JOptionPane.showMessageDialog(frame, "Nama tidak boleh kosong.");
            return;
        }
        tambahData(nama); // protected method parent
        txtNama.setText("");
        muatTabel();
    }

    private void aksiUbah() {
        int baris = tabel.getSelectedRow();
        String namaBaru = txtNama.getText().trim();
        if (baris < 0) {
            JOptionPane.showMessageDialog(frame, "Pilih data di tabel terlebih
dahulu.");
            return;
        }
        if (namaBaru.isEmpty()) {
            JOptionPane.showMessageDialog(frame, "Nama tidak boleh kosong.");
            return;
        }
        if (ubahData(baris + 1, namaBaru)) {
            txtNama.setText("");
            muatTabel();
        }
    }
```

```
    private void aksiHapus() {
        int baris = tabel.getSelectedRow();
        if (baris < 0) {
            JOptionPane.showMessageDialog(frame, "Pilih data di tabel terlebih
dahulu.");
            return;
        }
        String nama = modelTabel.getValueAt(baris, 1).toString();
        int jawab = JOptionPane.showConfirmDialog(frame,
                "Hapus data '" + nama + "'?", "Konfirmasi",
                JOptionPane.YES_NO_OPTION);

        if (jawab == JOptionPane.YES_OPTION) {
            if (hapusData(baris + 1)) { // protected method parent
                txtNama.setText("");
                muatTabel();
            }
        }
    }

    public void tampilkan() {
        frame.setVisible(true);
    }

    public static void main(String[] args) {
        SwingUtilities.invokeLater(() -> new MahasiswaGUI().tampilkan());
    }
}
```

5.1 Kerangka class

```
package gui;

import induk.Mahasiswa;
import javax.swing.*;
import javax.swing.table.DefaultTableModel;

public class MahasiswaGUI extends Mahasiswa {

    private JFrame frame;
    private JTextField txtNama;
    private JTable tabel;
    private DefaultTableModel modelTabel;

    public MahasiswaGUI() {
        buatTampilan();
        muatTabel();
    }
    // ...
}
```

Constructor memanggil dua hal secara berurutan: menyusun tampilan, lalu mengisi tabel dari data yang ada.

5.2 Menyusun tampilan dengan setBounds

```
frame = new JFrame(getNamaApp());
frame.setSize(500, 440);
frame.setLocationRelativeTo(null);
frame.setResizable(false);
frame.setLayout(null);

JLabel lblNama = new JLabel("Nama:");
lblNama.setBounds(20, 20, 60, 25);
frame.add(lblNama);
```

setBounds menerima empat angka: posisi x, posisi y, lebar, dan tinggi, semuanya dalam piksel dengan titik awal di pojok kiri atas. Pengaturan ini hanya berfungsi jika layout bawaan dimatikan dengan setLayout(null). Karena posisi bersifat tetap, ukuran jendela dikunci dengan setResizable(false).

Rincian posisi komponen:

Komponen	x	y	lebar	tinggi
Label Nama	20	20	60	25
TextField	80	20	380	25
Tombol Tambah	80	60	100	30
Tombol Ubah	190	60	100	30
Tombol Hapus	300	60	100	30
Grid (JScrollPane)	20	110	440	270

5.3 Grid dengan JTable

```
modelTabel = new DefaultTableModel(new String[]{"No", "Nama Mahasiswa"}, 0) {
    @Override
    public boolean isCellEditable(int row, int column) {
        return false;
    }
};
tabel = new JTable(modelTabel);
tabel.setSelectionMode(ListSelectionModel.SINGLE_SELECTION);

JScrollPane scroll = new JScrollPane(tabel);
scroll.setBounds(20, 110, 440, 270);
frame.add(scroll);
```

- JTable tidak menyimpan data sendiri. Data berada di DefaultTableModel, dan kolom harus ditentukan lebih dahulu saat model dibuat.
- Argumen 0 pada konstruktor model berarti tabel dimulai tanpa baris.

- Method isCellEditable dibuat mengembalikan false supaya sel tidak bisa diketik langsung. Pengubahan data dilakukan lewat textfield dan tombol Ubah.
- JTable perlu dibungkus JScrollPane agar muncul scrollbar dan header kolom. Yang diberi setBounds adalah JScrollPane-nya, bukan JTable.

5.4 Menghubungkan tabel dengan data class induk

```
private void muatTabel() {
    modelTabel.setRowCount(0);
    for (int i = 0; i < daftarNama.size(); i++) {
        modelTabel.addRow(new Object[]{i + 1, daftarNama.get(i)});
    }
}
```

Method ini mengosongkan tabel, lalu mengisinya kembali dari daftarNama, yaitu field protected milik Mahasiswa. Method ini dipanggil setiap kali data berubah, sehingga tabel selalu mengikuti isi ArrayList. Sumber kebenaran data tetap ada di class induk, sedangkan tabel hanya cermin tampilannya.

5.5 Operasi CRUD pada GUI

```
private void aksiTambah() {
    String nama = txtNama.getText().trim();
    if (nama.isEmpty()) {
        JOptionPane.showMessageDialog(frame, "Nama tidak boleh kosong.");
        return;
    }
    tambahData(nama);
    txtNama.setText("");
    muatTabel();
}
```

Pola yang sama dipakai pada semua operasi: ambil input, validasi, panggil method protected milik induk, bersihkan textfield, lalu muat ulang tabel.

Pada aksiUbah dan aksiHapus, baris yang dipilih diambil dengan tabel.getSelectedRow(). Nilainya dimulai dari 0 dan bernilai -1 bila tidak ada baris terpilih. Karena method induk memakai nomor yang dimulai dari 1, pemanggilannya ditulis ubahData(baris + 1, namaBaru) dan hapusData(baris + 1).

Pada aksiHapus, program menampilkan dialog konfirmasi dengan JOptionPane.showConfirmDialog sebelum data dihapus. Kebiasaan meminta konfirmasi untuk operasi yang tidak bisa dibatalkan adalah praktik yang baik pada aplikasi apa pun.

5.6 Event

```
btnTambah.addActionListener(e -> aksiTambah());
btnUbah.addActionListener(e -> aksiUbah());
btnHapus.addActionListener(e -> aksiHapus());
```

Setiap tombol dihubungkan ke satu method dengan lambda. Selain itu ada dua event tambahan pada tabel:

- ListSelectionListener, yang menyalin nama pada baris terpilih ke textfield agar siap diedit. Pengecekan getValueIsAdjusting() mencegah kode dijalankan dua kali saat pemilihan baris masih berlangsung.
- KeyListener, yang memanggil aksiHapus() ketika tombol Delete pada keyboard ditekan.

5.7 Menjalankan GUI

```
public static void main(String[] args) {
    SwingUtilities.invokeLater(() -> new MahasiswaGUI().tampilkan());
}
```

SwingUtilities.invokeLater memastikan jendela dibuat pada thread khusus Swing (Event Dispatch Thread). Ini cara standar memulai aplikasi Swing.

Catatan kecil: method tambahData di class induk masih mencetak pesan ke console. Pada versi GUI, pesan itu tidak terlihat oleh pengguna dan hanya muncul di jendela console atau output IDE. Ini tidak mengganggu, tetapi perlu diketahui agar tidak membingungkan saat membaca output.

6. Rangkuman

- protected membuka akses bagi class turunan, bahkan di package yang berbeda, tetapi menutup akses bagi class lain di luar package.
- Di luar package, akses protected hanya sah lewat objek dari class turunan itu sendiri.
- Data dan logika disimpan di class induk, sedangkan tampilan, baik console maupun grafis, ditaruh di class turunan.
- Karena Java hanya mengizinkan satu pewarisan, komponen seperti JFrame dipakai sebagai field.
- Satu class induk dapat dipakai oleh beberapa jenis tampilan tanpa menyalin logika CRUD.

Glosarium

Istilah	Pengertian
ActionListener	Antarmuka yang dipanggil ketika sebuah aksi terjadi, misalnya tombol diklik.
Class induk (superclass)	Class yang diwarisi oleh class lain. Pada modul ini adalah Mahasiswa.
Class turunan (subclass)	Class yang mewarisi class lain memakai extends.
CRUD	Singkatan Create, Read, Update, Delete: empat operasi dasar pengelolaan data.
DefaultTableModel	Model data bawaan Swing yang menyimpan kolom dan baris untuk JTable.
Default (package-private)	Tingkat akses ketika tidak ada modifier ditulis, hanya terlihat di package yang sama.
Encapsulation	Prinsip menyembunyikan data internal dan membuka akses hanya lewat jalur yang ditentukan.
Event Dispatch Thread	Thread khusus yang mengurus seluruh tampilan dan event pada Swing.
Extends	Kata kunci untuk membuat sebuah class mewarisi class lain.
Inheritance (pewarisan)	Mekanisme sebuah class memakai field dan method milik class lain.
JFrame	Class Swing untuk jendela utama aplikasi.
JScrollPane	Wadah yang menambahkan scrollbar pada komponen di dalamnya.
JTable	Komponen Swing untuk menampilkan data dalam bentuk grid baris dan kolom.
JTextField	Komponen Swing untuk mengisi teks satu baris.
Lambda	Penulisan ringkas untuk fungsi tanpa nama, misalnya e -> aksiTambah().
setBounds	Method untuk menentukan posisi dan ukuran komponen secara manual.
Swing	Pustaka Java untuk membuat antarmuka grafis desktop.

Daftar Pustaka

Bloch, J. (2018). Effective Java (3rd ed.). Addison-Wesley.

Deitel, P., & Deitel, H. (2018). Java How to Program, Early Objects (11th ed.). Pearson.

Gosling, J., Joy, B., Steele, G., Bracha, G., Buckley, A., Smith, D., & Bierman, G. (2023). The Java Language Specification, Java SE 21 Edition. Oracle. Bagian 6.6 Access Control.

Horstmann, C. S. (2022). Core Java, Volume I: Fundamentals (12th ed.). Pearson.

Oracle. (n.d.). Controlling Access to Members of a Class. The Java Tutorials. https://docs.oracle.com/javase/tutorial/java/javaOO/accesscontrol.html

Oracle. (n.d.). How to Use Tables. The Java Tutorials, Creating a GUI With Swing. https://docs.oracle.com/javase/tutorial/uiswing/components/table.html

Sierra, K., & Bates, B. (2022). Head First Java (3rd ed.). O'Reilly Media.